

U

O

W

CSIT314/ISIT950

Systems Development Methodologies

Verification & Validation and Test-driven development



UNIVERSITY
OF WOLLONGONG
AUSTRALIA

Software Development Activities

- Software Engineering plays a critical part in the software development
 - Planning
 - Requirements analysis
 - Design
 - Implementation
 - **Verification & Validation**
 - Maintenance and evolution



Verification & Validation

- **Verification:**
 - "Are we building the product right?"
 - The software should conform to its specification (such as non-functional requirement).
- **Validation:**
 - "Are we building the right product?"
 - The software should do what the user really requires.



Verification & Validation

- Is a whole life-cycle process
 - Can be applied during each phase
- Two principal objectives
 - To discover and rectify defects/bugs in a system
 - To assess whether the system is usable in an operational situation
- Mainly focus on 1st V

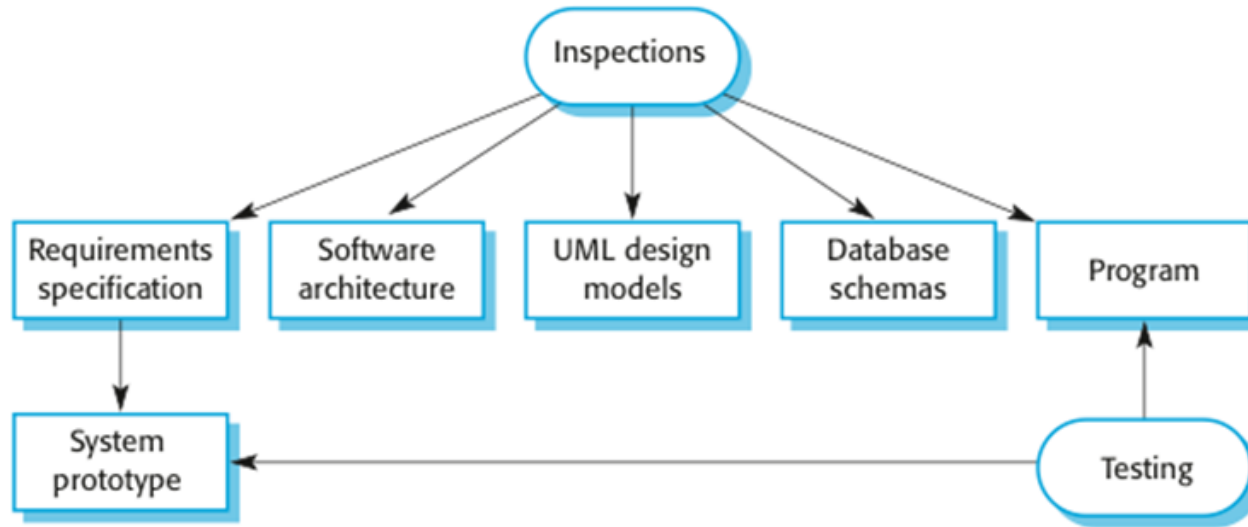


Static and Dynamic Verification

- Software inspections
 - analysis of the static system representation to discover problems (static verification)
- Software testing
 - executing and observing product behaviour (dynamic verification)
 - The system is executed with test data and its operational behaviour is observed



Static and Dynamic Verification



Software inspections

- Designs and code are reviewed by people other than the original developer.
- Inspections do not require execution of a system so may be used before implementation.
 - examine the source representation with the aim of discovering anomalies and defects
 - discovering program errors
- Applied to any representation of the system
 - requirements, design, configuration data, test data, etc.



Software inspections (example)

- **Variable and Constant Declaration Defects (VC)**
 - naming conventions, non-local variables, constants vs variables
- **Function Definition Defects**
 - function parameter value checked before being used
- **Class Definition Defects**
 - constructor and destructor, memory leak, class inheritance hierarchy
- **Control Flow Defects**
 - loop terminate, nesting of loops and branches
- **Input-Output Defects**
- **Comment Defects**
- **others**



Advantages of inspections

- During testing, errors can hide other errors.
- Incomplete versions of a system can be inspected.
- An inspection can also consider broader quality attributes of a program, such as compliance with standards, portability and maintainability.
- With the reuse domain and programming knowledge, experienced reviewers are likely to have seen the types of error that commonly arise.
 - Knowledge sharing



Advantages of inspections

- Reviews detect (and correct) defects early in development.
 - early detect leads to a cost-effective correction
 - reduce overall development effort
- **Inspections vs. Testing**
 - They are complementary with each other and not opposing
 - properties hard to test
 - security, exception handling
 - Requirements, architecture, design documents (before testing)
 - not for executing as tests



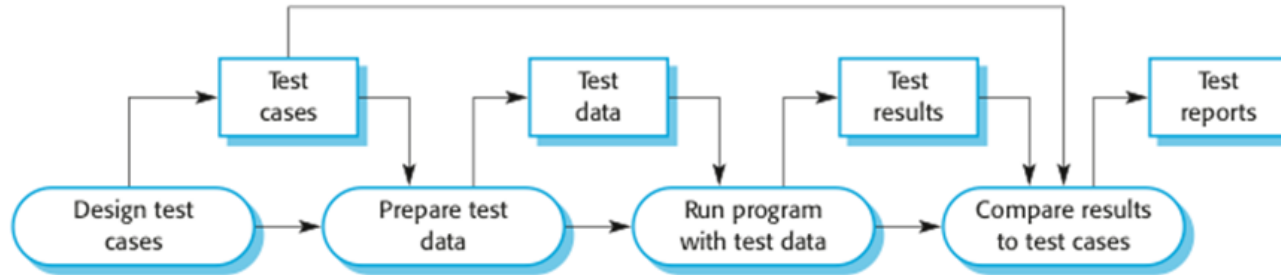
Software Testing

- Executing a program in order to
 - **force a program to work incorrectly**
 - Find what is wrong
 - **demonstrate that the program works correctly**
 - fault-based testing: target on certain types of faults
 - for example, divide by zero => the test data would include zero.
- Typically, testing takes about 30-40% of budget for system development



Software Testing

- Testing consists of a set of test cases which are systematically planned and executed



Test Case Design

- Each test case is uniquely identified or created
- Purpose of each test case should be stated
- Expected results are listed in the test case
 - include expected outputs, resultant changes and error messages



A Standard Test Case Matrix

- Case ID
- Scenario
- Steps
- Prerequisites
- Test Data
- Expected Results
- Actual Results
- Test Status – Pass/Fail

- **Test Case ID:** #BST001
- **Test Scenario:** To authenticate a successful user login
- **Test Steps:**
 - The user navigates xxx website.
 - The user enters a registered email address as username
 - The user enters the registered password.
 - The user clicks 'Sign In.'
- **Prerequisites:**
 - A registered ID with a unique username and password.
 - Browser: Chrome v 86++
 - Device: Samsung Galaxy Tab S7.
- **Test Data:** Legitimate username and password.
- **Expected/Intended Results:** Once username and password are entered, the web page redirects to the user's profile page
- **Actual Results:** As Expected
- **Test Status – Pass/Fail:** Pass



Who performs testing?

- Testing is conducted by two (or three) groups:
 - the software developer
 - (for large projects) an independent test group
 - the customer



Development testing

- Development testing includes all testing activities that are **carried out by the team developing the system**
 - Unit testing
 - individual units or object classes are tested
 - focus on testing the functionality of objects or methods
 - System testing as a whole
 - testing component **interactions**



Unit testing

- Unit testing is the process of testing individual components in isolation
- Units may be:
 - Individual functions or methods within an object
 - Object class as a whole
 - Testing all operations associated with an object
 - Method A * Method B
 - Setting and interrogating all object attributes
 - Exercising the object in all possible states



Unit testing (example)

- Need to define test cases for reportWeather, reportStatus, powerSave, remoteControl, reconfigure, restart and shutdown.
- Using a state model, identify sequences of state transitions to be tested and the event sequences to cause these transitions
 - Shutdown -> Running-> Shutdown
 - Configuring-> Running-> Testing -> Transmitting -> Running

Weather station

state

reportWeather()
reportStatus()
powerSave()
remoteControl()
reconfigure()
restart()
shutdown()



Automated unit testing

- Unit testing can/should be automated so that tests are run and checked without manual intervention
 - a test automation framework (such as JUnit or cppUnit)
 - generic test classes (e.g. `cppUnit::TestFixture`) is to be extended to create specific test cases.
 - run all tests, often through some GUI, to report the success



Unit test effectiveness

- Outcome: when used as expected, the component that you are testing does what it is supposed to do.
- Two types of unit test cases:
 - normal operation of a program and so the component works as expected
 - use abnormal inputs to check that these are properly processed and do not crash the component



Testing Techniques

- **White box Testing**
 - examining the internal workings (i.e. source code) of each module
- **Black box Testing**
 - without knowing the internal workings of the programs
 - a set of results is derived to ensure that modules produce correct results.



White Box Testing

- White Box tests **focus on the program control structure**

- **Applied to increase logic coverage**

- Statement coverage
- Decision (branch) coverage
- Condition coverage
- Path coverage

```
premium = 500;  
if (age < 25) && (sex == male) && !married {  
    premium += 500;  
}  
else {  
    if (married) || (sex == female)  
        premium -= 200;  
    if (age > 45) && (age < 65)  
        premium -= 100;  
}
```



White Box Testing

- Statement coverage
 - each statement is executed at least once
- Decision (branch) coverage
 - each statement ...; each decision takes on all possible outcome at least once
- Condition coverage
 - each statement...; each decision ...; each condition in a decision takes on all possible outputs at least once
- Path coverage
 - each statement ...; all possible combinations of condition outcomes in each decision occur at least once



Black Box Testing

- Without knowing the internal works
- **Mainly from inputs and with two techniques:**
 - Random (uniform):
 - Pick possible inputs uniformly, treats all inputs as equally valuable
 - **Avoids bias**
 - test designer can make the same logical mistakes and bad assumptions as the program designer (especially if they are the same person)
 - Drawback: impossible to perform testing for each set of test data, especially when there is a large pool of input combinations



Black Box Testing strategies

- Systematic (non-uniform):
 - Try to select important inputs (with specially valuable)
 - Usually by choosing representatives of classes that are apt to fail often or not at all
- Select test cases intelligently from the pool of test-case:
 - Equivalence Partitioning
 - Boundary Value Analysis

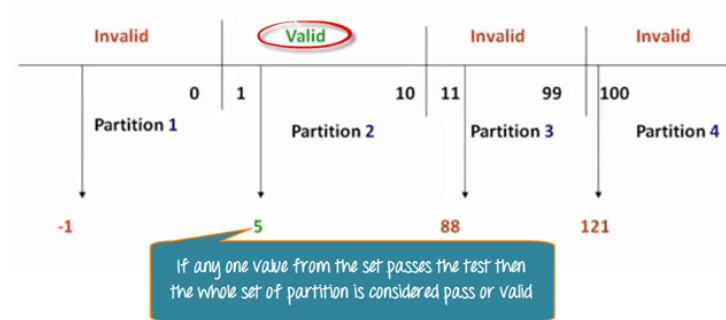


Equivalence Partitioning

- Divides the input data of software into different equivalence data classes.
 - pick only one value from each partition for testing
 - if passes, we assume this partition will also pass.

Example:

- Pizza values 1 to 10 is considered valid. A success message is shown.
- While value 11 to 99 are considered invalid for order and an error message will appear, "Only 10 Pizza can be ordered".



Boundary Value Analysis

- Selects test cases at the edges of the set or boundary values.
- Boundary Value Analysis is also called **range checking**
- Complements equivalence partitioning



In BOUNDARY VALUE ANALYSIS you will check the boundary values like 0, 1, 10, 11, 99, 100



Integration testing

- Tests on integrated components (subsystems/functions)
- Usually unit testing first, then integration testing will be offered to look at primarily interface faults.
 - Testing effort should focus on interfaces between units rather than their internal details.



Systems Testing

- This test encapsulates the environment as well as the product:
 - Function Testing
 - tests functional requirements of the system
 - Performance Testing
 - tests non-functional requirements, e.g. reliability, availability, etc.
 - Acceptance Testing
 - performs validation testing on the system prior to handover to the customers (client satisfaction)



Test-driven development

- A bridge between testing and coding
- Tests are written before code and 'passing' the tests is the critical driver of development.
- Develop code incrementally, along with a test for that increment.
 - **don't move to the next increment until the current code passes its test.**



Test-driven development (steps)

- Start by identifying the new functionality that is required. This should normally be small and implementable in a few lines of code
- Write a test for this functionality and implement this as an automated test
- Run the test, along with all other tests that have been implemented. Initially, you have not implemented the functionality so the new test will fail.
- Implement the functionality and re-run the test.
- Once all tests run successfully, you move on to implementing the next chunk of functionality.



Benefits of test-driven development

- Code coverage
 - Every code segment that you write has at least one associated test so all code written has at least one test.
- Regression testing
 - developed incrementally to make sure changes have not 'broken' previously working code.
- Simplified debugging
 - When a test fails, it should be obvious where the problem lies. The newly written code needs to be checked and modified.



Tools

- `cpputest`
- `csUnit (.Net)`
- `Cunit`

- `HTMLUnit`
- `HTTPUnit`
- `JUnit`
- `PHPUnit`

- `PyUnit (Python)`
- `DocTest (Python)`
- `TestOoB (Python)`

- `Test::Unit (Ruby)`
- `VBUnit`

